LoRA: Low-Rank Adaptation on RoBERTa-base / SST-2

Elliott Lin, Anne Xia, Trisha Saini, Samarth Shridhar · CS 4782, Cornell University · Spring 2026

1. Introduction

Updating large language models for new downstream tasks through full fine-tuning is expensive. Every parameter must be retrained, every task needs a separate checkpoint, and the compute, storage, memory, and time requirements make experimentation costly. Our project addresses how to fine-tune a pretrained model efficiently while still successfully completing new tasks, adapting a pretrained model to a new task without retraining the entire model.

We reimplemented ideas from *LoRA: Low-Rank Adaptation of Large Language Models* by Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, and Chen [1]. The paper's main contribution is a parameter-efficient fine-tuning method that freezes the original pretrained model weights and adds small trainable low-rank matrices. LoRA learns updates to the low-rank matrices, reducing the number of trainable parameters and computational cost while trying to achieve the same performance as full fine-tuning.

2. Chosen Result

We aimed to reproduce the core claim of the LoRA paper: that low-rank adaptation can approach full fine-tuning performance while using far fewer trainable parameters. Specifically, we reproduce the RoBERTa-base row of **Table 2** in [1], which reports **95.1%** validation accuracy for LoRA at rank 8 (0.3M trainable parameters) versus **94.8%** for full fine-tuning (125.0M trainable parameters) on the Stanford Sentiment Treebank v2 (SST-2). We chose this result because it is the cleanest demonstration of the paper's headline claim on a model size we could realistically train within our compute limits.

We also extend the paper with a joint sweep over rank $r \in \{2, 4, 6, 8\}$ and scaling factor $\alpha \in \{2, 4, 6, 8\}$ to investigate how the α/r ratio affects training stability, an interaction the paper does not analyze in detail.

3. Methodology

We used HuggingFace's `FacebookAI/roberta-base` as our pretrained model and `stanfordnlp/sst2` as our dataset (67,349 train / 872 validation; the public test split has hidden labels, so following the GLUE convention we report on validation). We implemented our own LoRA adapter module that wraps a frozen `nn.Linear` and adds two trainable matrices A (Gaussian init, std $1/\sqrt{r}$) and B (zero init), so the LoRA contribution is exactly zero at step 0. We patched this adapter onto the query, key, and value projections in all 12 RoBERTa encoder layers (36 adapters total). The paper applies LoRA to query and value only; our setup is a strict superset.

Training used AdamW, cross-entropy loss, batch size 16, max sequence length 512, learning rate 5×10^{-4}, and 10 epochs (versus 30 to 60 in the paper). We logged training and validation loss per epoch, validation and test accuracy, and trainable parameter counts, and added checkpointing on best validation loss. Training ran on Google Colab A100 GPUs. Beyond the headline configuration ($r = \alpha = 8$), we ran three additional experiments: (i) holding $\alpha = 8$ fixed while varying rank, (ii) holding $\alpha/r = 1$ fixed while varying rank and α, and (iii) holding $r = 2$ fixed while varying α.

4. Results & Analysis

Main result. Our best run ($r = \alpha = 8$) reached **94.6% validation accuracy** and **90.9% test accuracy** on SST-2, compared to the paper's reported 95.1% LoRA validation accuracy and 94.8% full fine-tuning validation accuracy. Our validation result closely matches the paper. The test accuracy is lower, which we attribute primarily to the shortened training schedule: at epoch 10 our training loss was still consistently decreasing, suggesting the model had not converged. For $r = 6$ with $\alpha = 8$, validation accuracy reached 94.1%, also approaching the paper's full fine-tuning baseline. Rank 4

captured most of the classification signal at 83.7% but trailed r = 6, 8, and rank 2 failed to learn meaningful classification (55.1%).

The α/r interaction. We hypothesized that the rank-2 and rank-4 failures were a scaling issue rather than a bottleneck-width issue. During the forward pass, the low-rank output is scaled by α/r. Holding $\alpha = 8$ fixed means the rank-2 update is scaled by 4 and the rank-4 update by 2, while rank-8 is scaled by 1, so at low rank the LoRA contribution is much larger than the frozen pretrained signal, which we suspect destabilizes training. We retrained r = 2, 4, 6 with $\alpha = r$ (so $\alpha/r = 1$ throughout). All three configurations recovered; in fact, validation and test accuracy *improved* as rank decreased. We believe this is because for a binary classification task with limited training signal, a tighter bottleneck forces the model to discard noise and focus on the discriminative features that distinguish the two classes.

Maximum stable α. To find the largest α/r ratio that still trains, we held $r = 2$ fixed and swept $\alpha \in \{2, 4, 6, 8\}$. Performance was strong at $\alpha = 2$ and $\alpha = 4$, and degraded sharply at $\alpha = 6$ and $\alpha = 8$. This gives a concrete practical rule from our sweep: $\alpha/r \leq 2$ appears to be the stability threshold on SST-2.

Deviations from the paper. Beyond the 10-epoch schedule, our setup differs from [1] in three ways. First, we apply LoRA to Q, K, and V rather than Q and V only (a strict superset of the paper's best configuration). Second, we report a single seed rather than the median over five. Third, we compare against the paper's reported full-FT number rather than training a baseline at matched epochs. However, none of these explain the test-accuracy gap, which is most likely a result of under-training.

5. Reflections

Our main takeaway is that the α/r scaling factor matters more than the paper's treatment suggests. The paper sets α equal to the first rank tried and moves on, framing this as a convenience that decouples learning rate from rank. Our experiments show that this convenience breaks down at the boundary: at fixed $\alpha = 8$, ranks 2 and 4 fail to train, but rescaling α to match each rank recovers performance and in some cases improves on $r = 8$. The practical rule we extract is to keep $\alpha/r \leq 2$.

A secondary lesson is about the meaning of "sufficient rank." The paper reports that very low ranks (r = 1, 2) work well for GPT-3 on tasks like WikiSQL and MultiNLI. Our finding that rank 2 trains successfully on SST-2 only when α is also small suggests that the effective rank of the adaptation depends jointly on r and the scaling factor, not on r alone. A practitioner choosing a low rank for storage or compute reasons cannot simply pick a small r and inherit the paper's headline conclusions; the α setting determines whether that low-rank adaptation is actually trainable.

Future directions: (1) train to convergence at $r = 8$ to close the test-accuracy gap with the paper's reported numbers; (2) sweep α/r at larger ranks (r = 16, 32) to test whether the $\alpha/r \leq 2$ rule generalizes; (3) extend to other GLUE tasks beyond SST-2 to check whether the "tighter bottleneck improves binary classification" observation holds for harder, multi-class tasks; and (4) implement the inference-time weight merging the paper describes (folding BA back into W_0) and verify the claimed zero added latency.

6. References

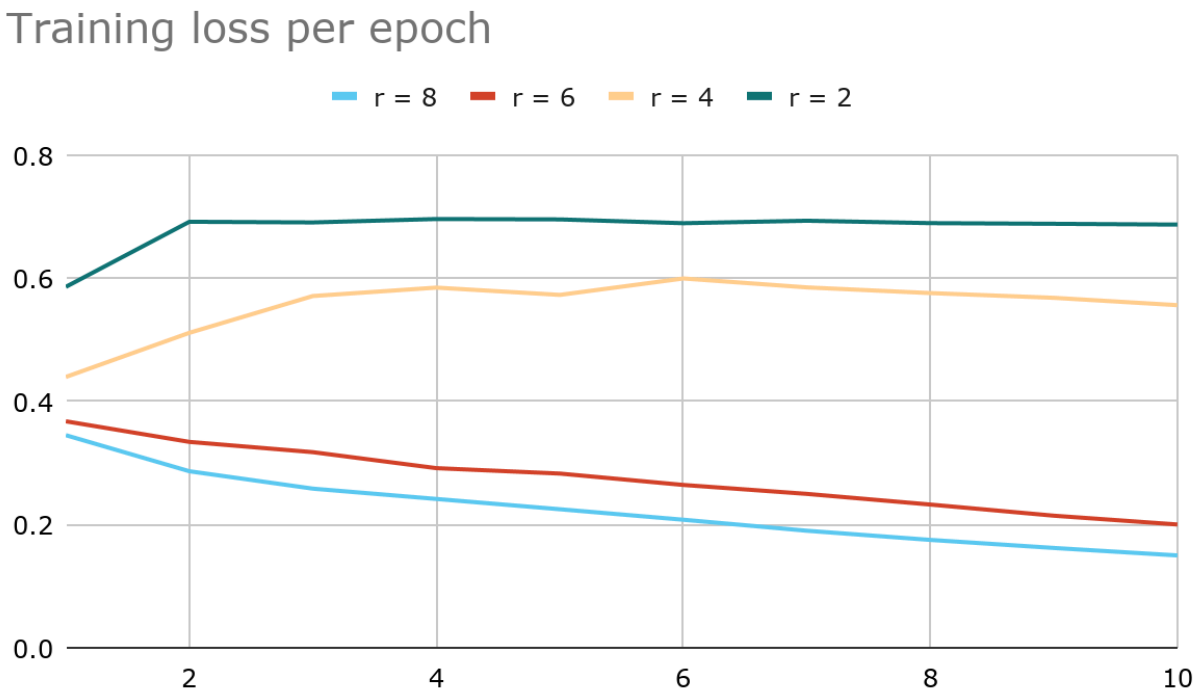
[1] Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685.

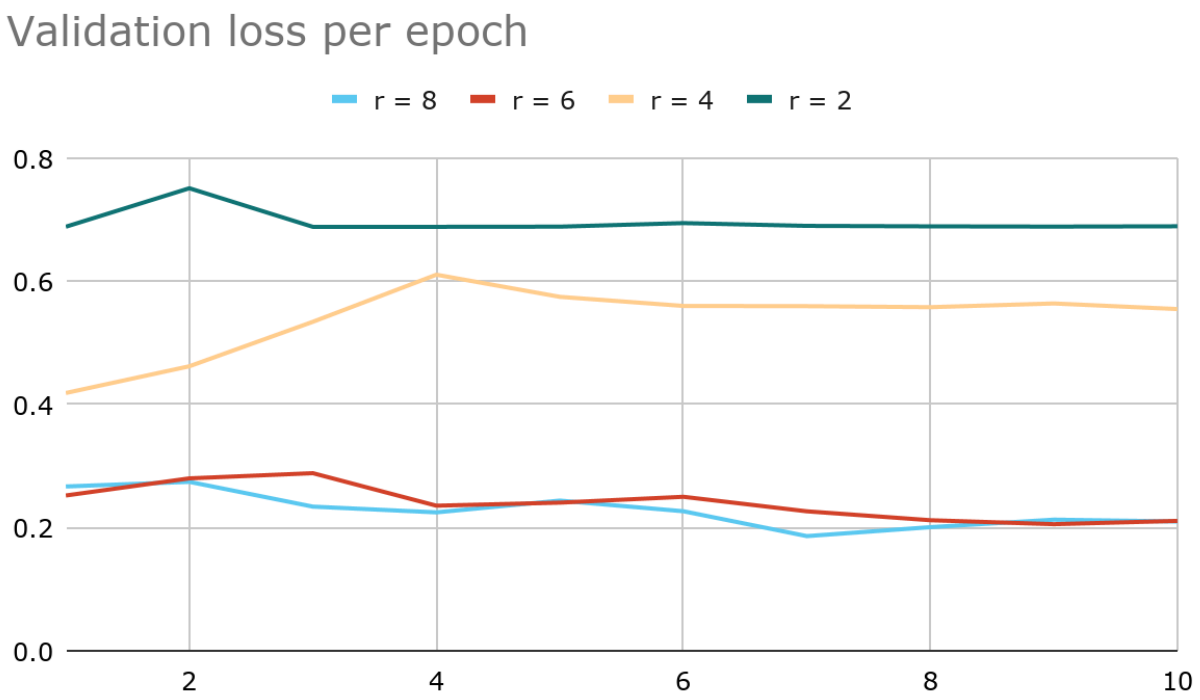
Table 2 from the original LoRA paper

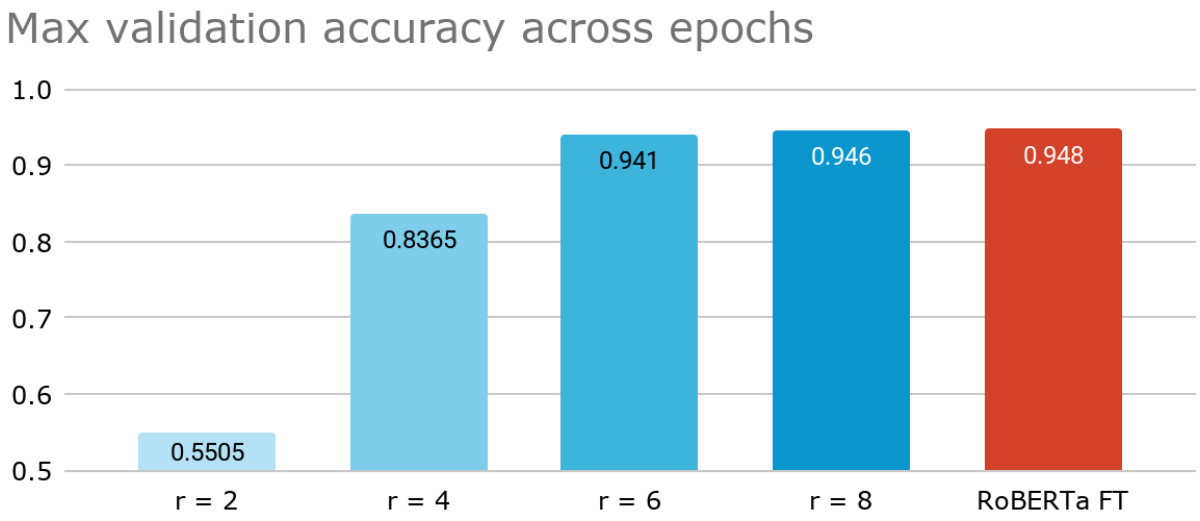
Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB_{base} (FT)*	125.0M	**87.6**	94.8	90.2	**63.6**	92.8	**91.9**	78.7	91.2	86.4
RoB_{base} (BitFit)*	0.1M	84.7	93.7	**92.7**	62.0	91.8	84.0	81.5	90.8	85.2
RoB_{base} (Adpt^{D})*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$	$88.5_{\pm1.1}$	$60.8_{\pm.4}$	$93.1_{\pm.1}$	$90.2_{\pm.0}$	$71.5_{\pm2.7}$	$89.7_{\pm.3}$	84.4
RoB_{base} (Adpt^{D})*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$	$88.4_{\pm.1}$	$62.6_{\pm.9}$	$93.0_{\pm.2}$	$90.6_{\pm.0}$	$75.9_{\pm2.2}$	$90.3_{\pm.1}$	85.4
RoB_{base} (LoRA)	0.3M	$87.5_{\pm.3}$	$\mathbf{95.1}_{\pm.2}$	$89.7_{\pm.7}$	$63.4_{\pm1.2}$	$\mathbf{93.3}_{\pm.3}$	$90.8_{\pm.1}$	$\mathbf{86.6}_{\pm.7}$	$\mathbf{91.5}_{\pm.2}$	**87.2**
$\text{RoB}_{\text{large}}$ (FT)*	355.0M	90.2	**96.4**	**90.9**	68.0	94.7	**92.2**	86.6	92.4	88.9
$\text{RoB}_{\text{large}}$ (LoRA)	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.9}_{\pm1.2}$	$\mathbf{68.2}_{\pm1.9}$	$\mathbf{94.9}_{\pm.3}$	$91.6_{\pm.1}$	$\mathbf{87.4}_{\pm2.5}$	$\mathbf{92.6}_{\pm.2}$	**89.0**
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	3.0M	$90.2_{\pm.3}$	$96.1_{\pm.3}$	$90.2_{\pm.7}$	$\mathbf{68.3}_{\pm1.0}$	$\mathbf{94.8}_{\pm.2}$	$\mathbf{91.9}_{\pm.1}$	$83.8_{\pm2.9}$	$92.1_{\pm.7}$	88.4
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	0.8M	$\mathbf{90.5}_{\pm.3}$	$\mathbf{96.6}_{\pm.2}$	$89.7_{\pm1.2}$	$67.8_{\pm2.5}$	$\mathbf{94.8}_{\pm.3}$	$91.7_{\pm.2}$	$80.1_{\pm2.9}$	$91.9_{\pm.4}$	87.9
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	6.0M	$89.9_{\pm.5}$	$96.2_{\pm.3}$	$88.7_{\pm2.9}$	$66.5_{\pm4.4}$	$94.7_{\pm.2}$	$92.1_{\pm.1}$	$83.4_{\pm1.1}$	$91.0_{\pm1.7}$	87.8
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	0.8M	$90.3_{\pm.3}$	$96.3_{\pm.5}$	$87.7_{\pm1.7}$	$66.3_{\pm2.0}$	$94.7_{\pm.2}$	$91.5_{\pm.1}$	$72.9_{\pm2.9}$	$91.5_{\pm.5}$	86.4
$\text{RoB}_{\text{large}}$ (LoRA)†	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.2}_{\pm1.0}$	$68.2_{\pm1.9}$	$\mathbf{94.8}_{\pm.3}$	$91.6_{\pm.2}$	$\mathbf{85.2}_{\pm1.1}$	$\mathbf{92.3}_{\pm.5}$	**88.6**
DeB_{XXL} (FT)*	1500.0M	91.8	**97.2**	92.0	72.0	**96.0**	92.7	93.9	92.9	91.1
DeB_{XXL} (LoRA)	4.7M	$\mathbf{91.9}_{\pm.2}$	$96.9_{\pm.2}$	$\mathbf{92.6}_{\pm.6}$	$\mathbf{72.4}_{\pm1.1}$	$\mathbf{96.0}_{\pm.1}$	$\mathbf{92.9}_{\pm.1}$	$\mathbf{94.9}_{\pm.4}$	$\mathbf{93.0}_{\pm.2}$	**91.3**

Table 2: $\text{RoBERTa}_{\text{base}}$, $\text{RoBERTa}_{\text{large}}$, and $\text{DeBERTa}_{\text{XXL}}$ with different adaptation methods on the GLUE benchmark. We report the overall (matched and mismatched) accuracy for MNLI, Matthew's correlation for CoLA, Pearson correlation for STS-B, and accuracy for other tasks. Higher is better for all metrics. * indicates numbers published in prior works. † indicates runs configured in a setup similar to Houlsby et al. (2019) for a fair comparison.

Figure 1 - Results for alpha = 8 and rank = 2, 4, 6, 8






Max validation accuracy across epochs
1.0
0.9
0.8
0.7
0.6
0.5
0.5505
0.8365
0.941
0.946
0.948
r = 2
r = 4
r = 6
r = 8
RoBERTa FT

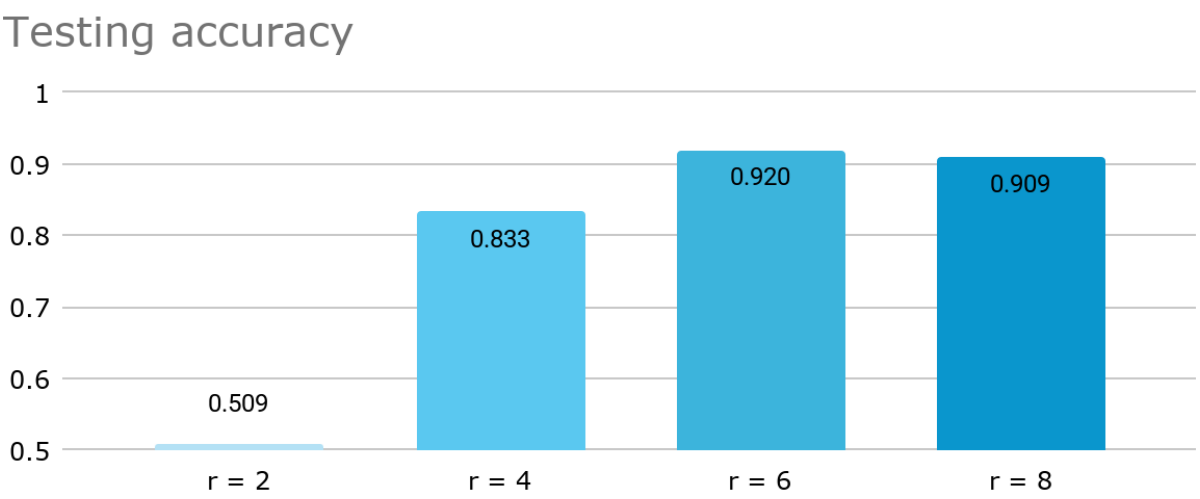
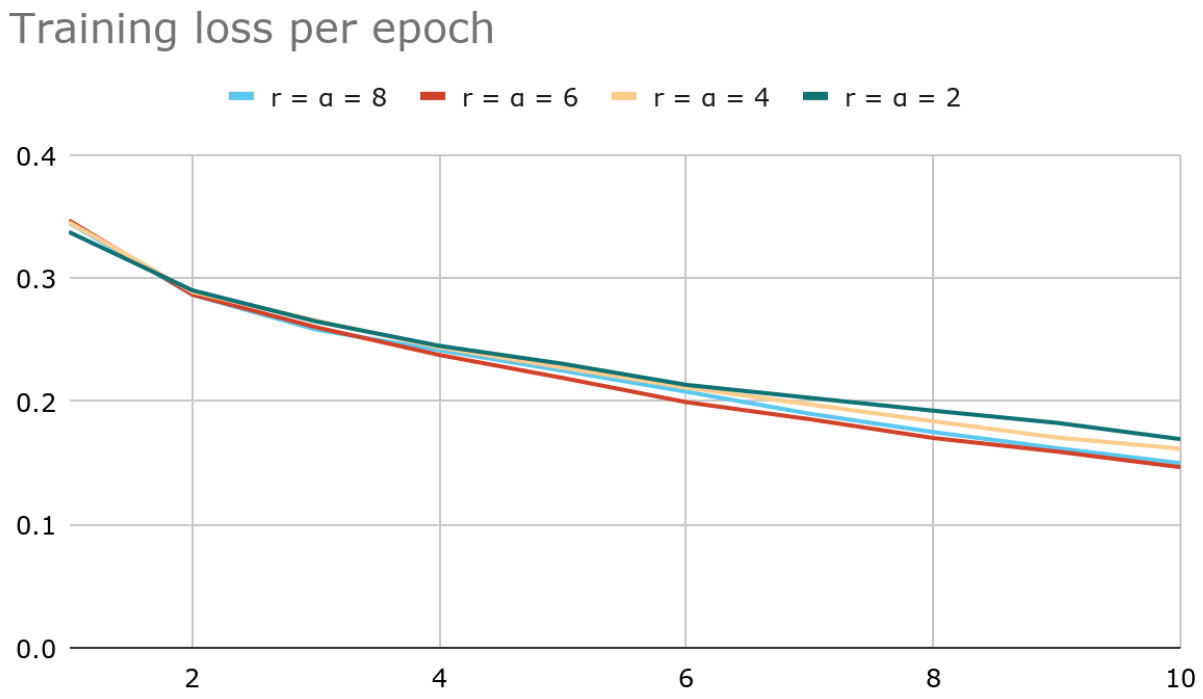
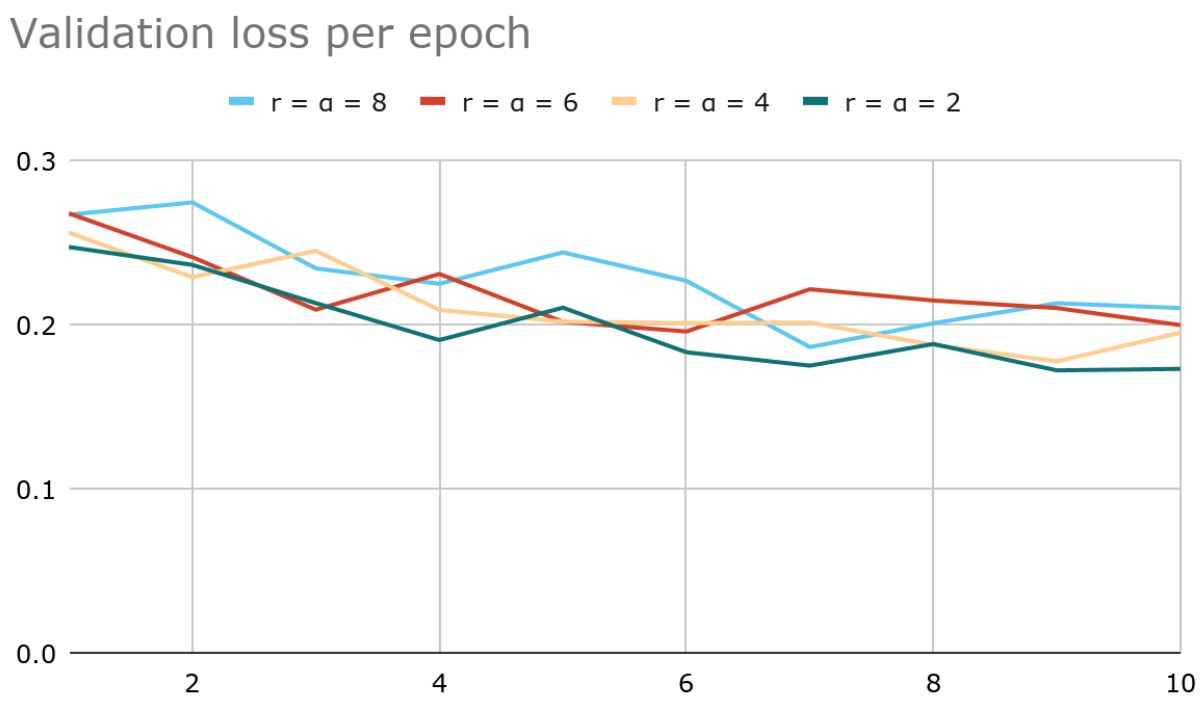
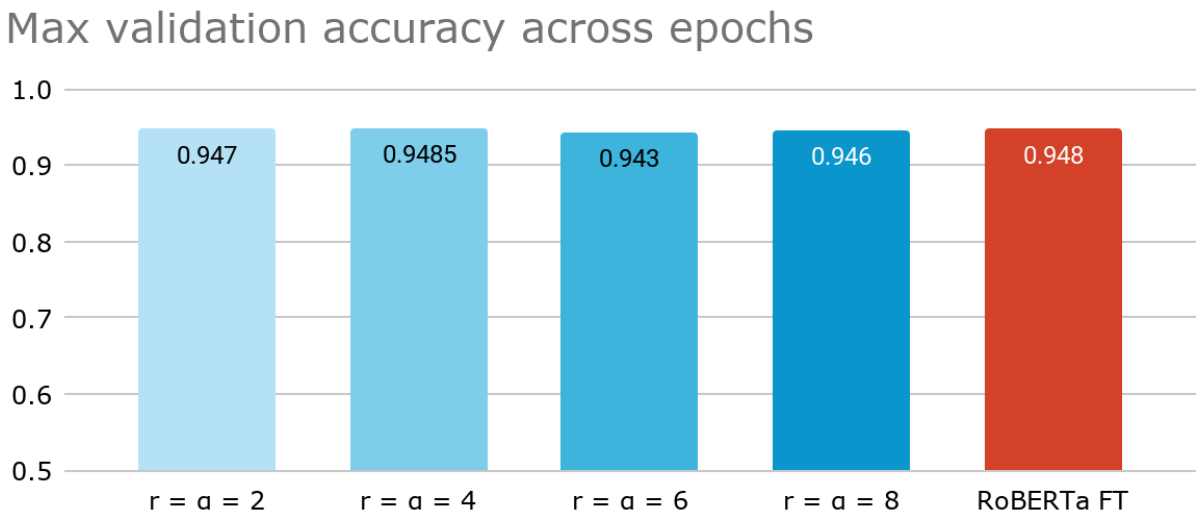

Testing accuracy
1
0.9
0.8
0.7
0.6
0.5
0.509
0.833
0.920
0.909
r = 2
r = 4
r = 6
r = 8

Figure 2 - Results for alpha = rank = 2, 4, 6, 8






Max validation accuracy across epochs
1.0
0.9
0.8
0.7
0.6
0.5
0.947
0.9485
0.943
0.946
0.948
r = a = 2
r = a = 4
r = a = 6
r = a = 8
RoBERTa FT

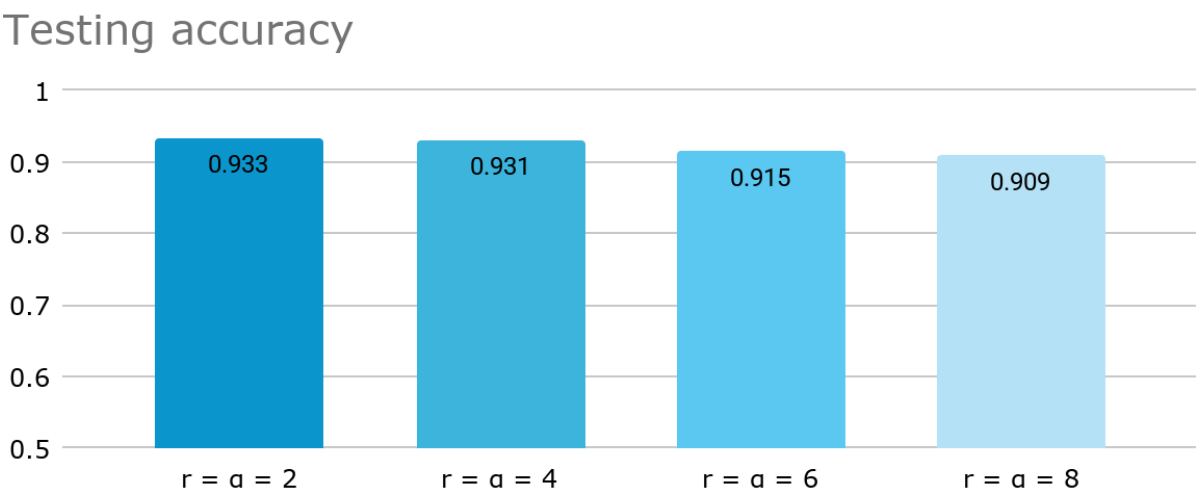
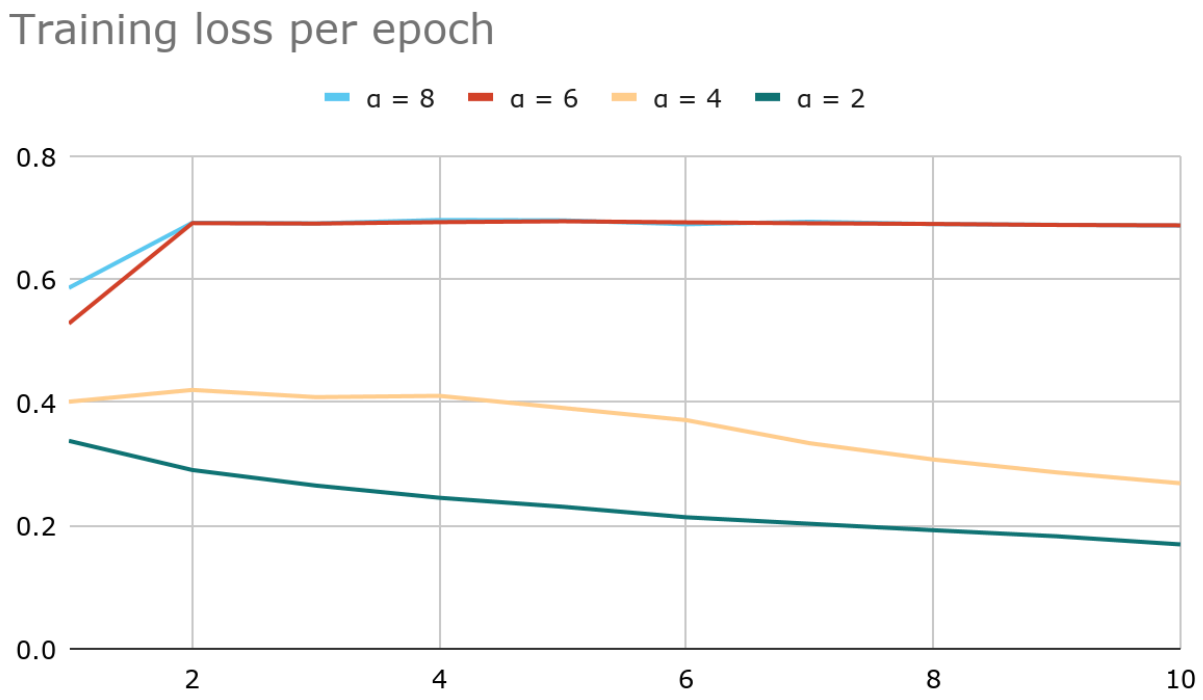
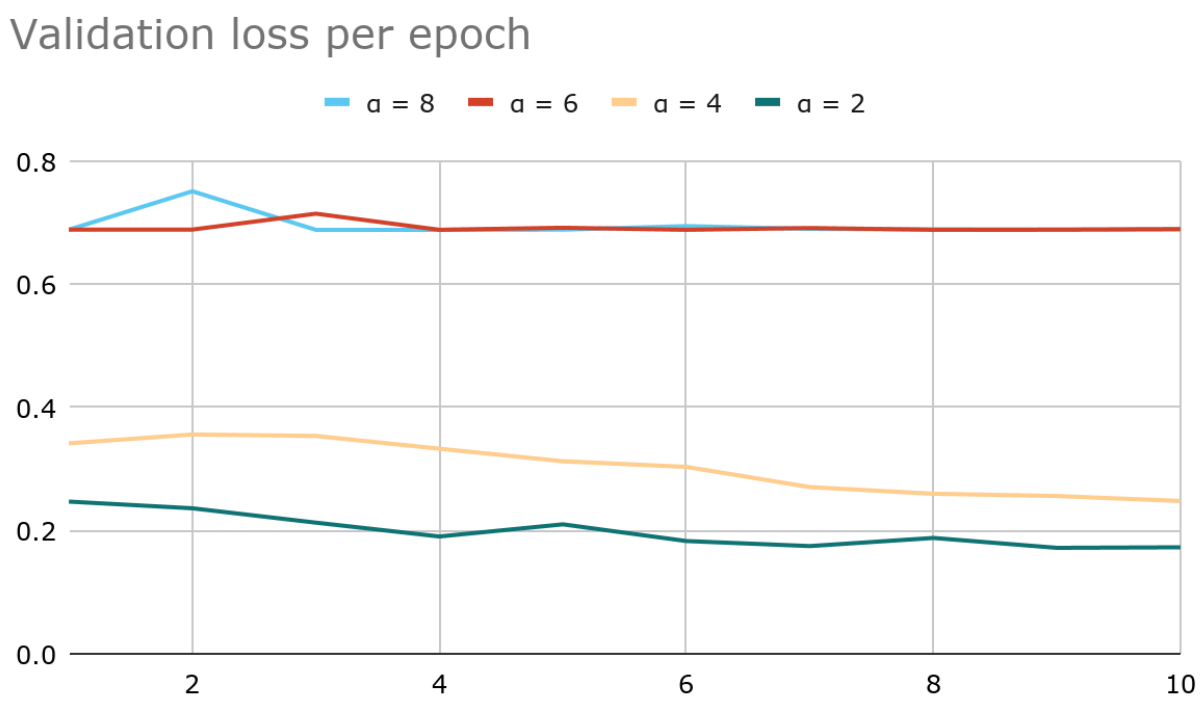
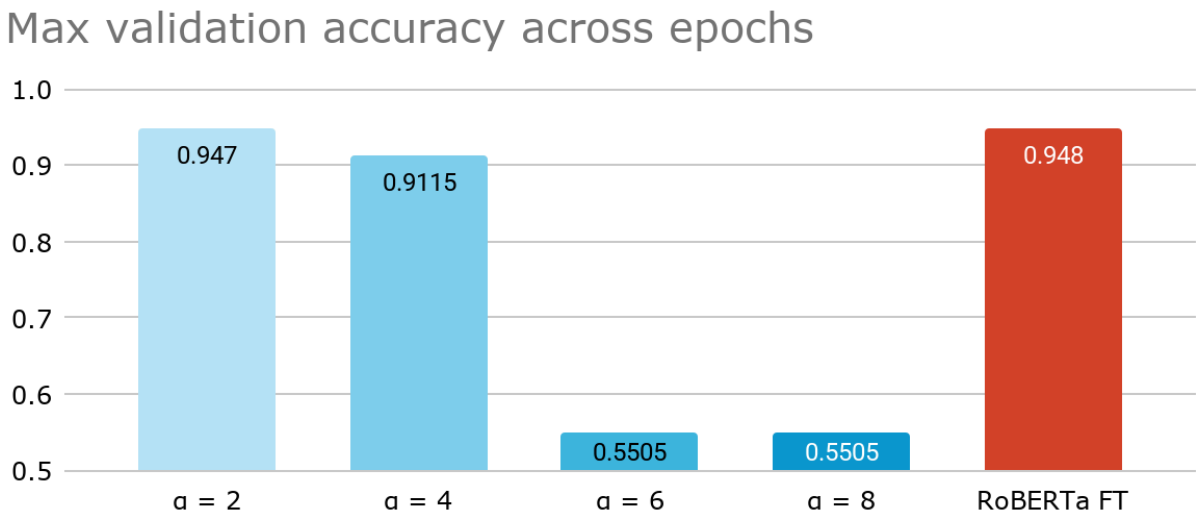
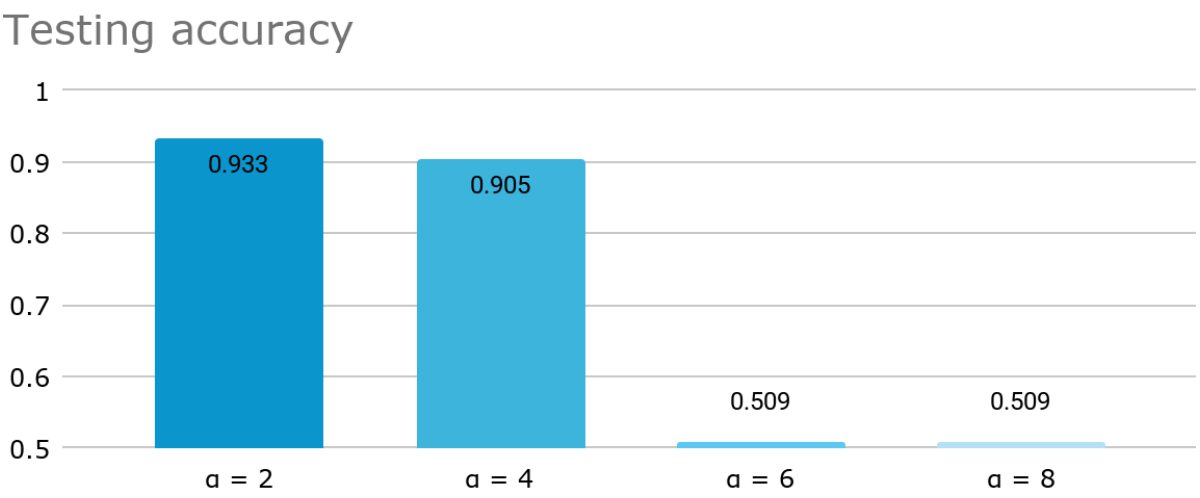

Testing accuracy
1
0.9
0.8
0.7
0.6
0.5
0.933
0.931
0.915
0.909
r = a = 2
r = a = 4
r = a = 6
r = a = 8

Figure 3 - Results for rank = 2 and alpha = 2, 4, 6, 8






Max validation accuracy across epochs
1.0
0.9
0.8
0.7
0.6
0.5
0.947
0.9115
0.5505
0.5505
0.948
α = 2
α = 4
α = 6
α = 8
RoBERTa FT


Testing accuracy
1
0.9
0.8
0.7
0.6
0.5
0.933
0.905
0.509
0.509
α = 2
α = 4
α = 6
α = 8